PYTHON AUTOMATION PROJECT

Inventory Automation

Turn an Excel inventory sheet into instant sales and low-stock reports, straight from the terminal.

AUTHOR	Othmane Qallidi
LANGUAGE	Python 3
LIBRARY	openpyxl
DATA SOURCE	Excel workbook (.xlsx)

01 / OVERVIEW

What does this project do?

Inventory Automation is a Python program that reads a company's inventory from an Excel file and answers two everyday business questions in seconds, without opening Excel or doing any manual calculation.

1	**Sales per product** Multiplies each product's price by the quantity sold and prints the total for every product.
2	**Low-stock warnings** Finds every product with fewer than 10 units left, so it can be restocked before it runs out.

How it works

Step	What happens
1. Load	The program opens the Excel workbook with `openpyxl` and selects the `Inventory` sheet.
2. Read	It loops through every row and reads the product name, price, remaining quantity and quantity sold.
3. Calculate	It computes `Price x Quantity Sold` for each product and flags any product with `Remaining Quantity < 10`.
4. Ask	A simple menu asks the user to type **1** (sales) or **2** (low stock).
5. Report	The matching report is printed in the terminal. Any other input shows a helpful error message.

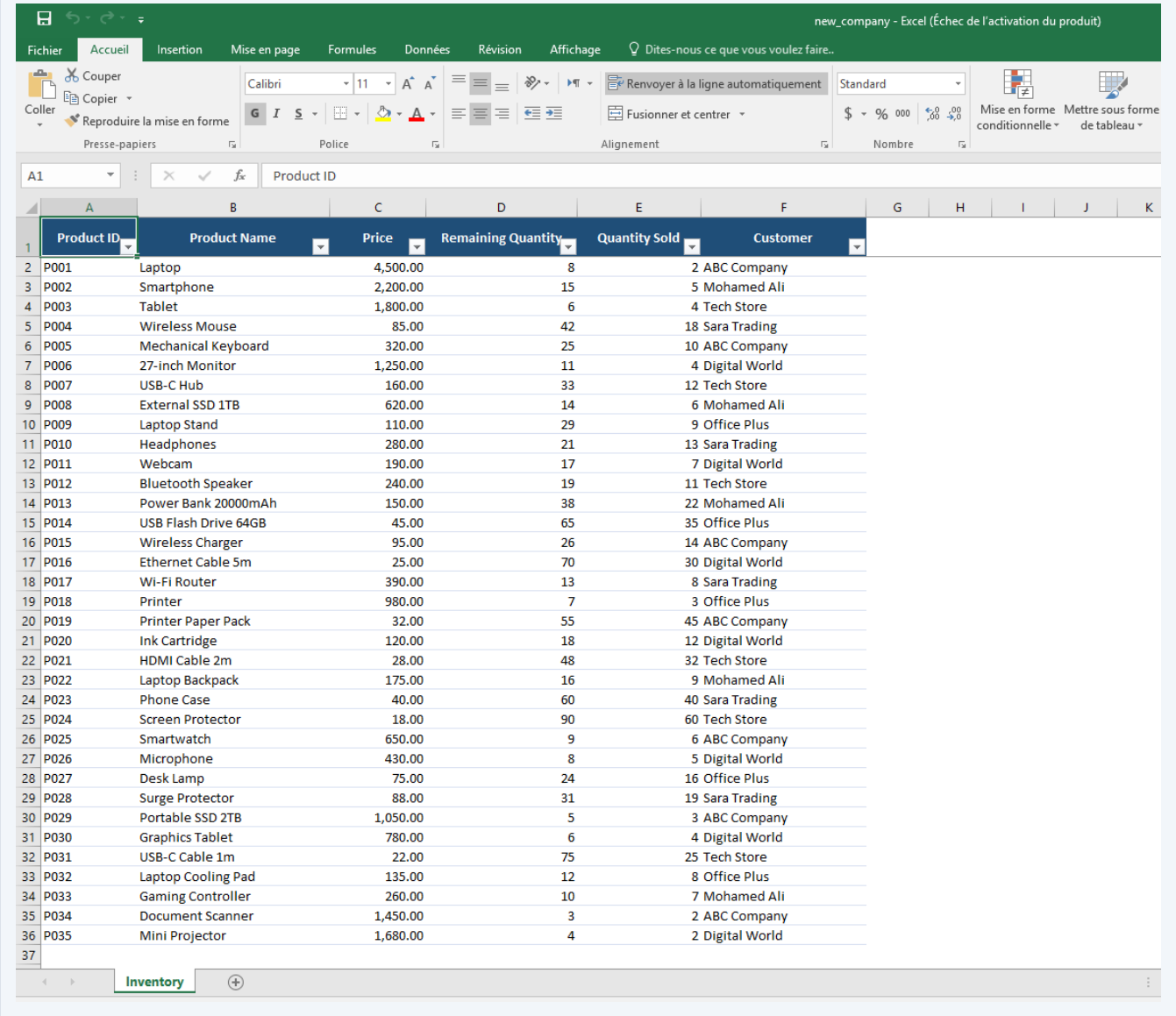
How to run it

- Install the library: `pip install openpyxl`
- Place your Excel file on your computer and set its path in `Automation.py`.
- Run the program: `python Automation.py`, then choose **1** or **2**.

02 / INPUT

The Excel inventory file

The program reads one workbook, `new_company.xlsx`, with a sheet called `Inventory`. It holds 35 products, each described by six columns.



Product ID	Product Name	Price	Remaining Quantity	Quantity Sold	Customer
P001	Laptop	4,500.00	8	2	ABC Company
P002	Smartphone	2,200.00	15	5	Mohamed Ali
P003	Tablet	1,800.00	6	4	Tech Store
P004	Wireless Mouse	85.00	42	18	Sara Trading
P005	Mechanical Keyboard	320.00	25	10	ABC Company
P006	27-inch Monitor	1,250.00	11	4	Digital World
P007	USB-C Hub	160.00	33	12	Tech Store
P008	External SSD 1TB	620.00	14	6	Mohamed Ali
P009	Laptop Stand	110.00	29	9	Office Plus
P010	Headphones	280.00	21	13	Sara Trading
P011	Webcam	190.00	17	7	Digital World
P012	Bluetooth Speaker	240.00	19	11	Tech Store
P013	Power Bank 20000mAh	150.00	38	22	Mohamed Ali
P014	USB Flash Drive 64GB	45.00	65	35	Office Plus
P015	Wireless Charger	95.00	26	14	ABC Company
P016	Ethernet Cable 5m	25.00	70	30	Digital World
P017	Wi-Fi Router	390.00	13	8	Sara Trading
P018	Printer	980.00	7	3	Office Plus
P019	Printer Paper Pack	32.00	55	45	ABC Company
P020	Ink Cartridge	120.00	18	12	Digital World
P021	HDMI Cable 2m	28.00	48	32	Tech Store
P022	Laptop Backpack	175.00	16	9	Mohamed Ali
P023	Phone Case	40.00	60	40	Sara Trading
P024	Screen Protector	18.00	90	60	Tech Store
P025	Smartwatch	650.00	9	6	ABC Company
P026	Microphone	430.00	8	5	Digital World
P027	Desk Lamp	75.00	24	16	Office Plus
P028	Surge Protector	88.00	31	19	Sara Trading
P029	Portable SSD 2TB	1,050.00	5	3	ABC Company
P030	Graphics Tablet	780.00	6	4	Digital World
P031	USB-C Cable 1m	22.00	75	25	Tech Store
P032	Laptop Cooling Pad	135.00	12	8	Office Plus
P033	Gaming Controller	260.00	10	7	Mohamed Ali
P034	Document Scanner	1,450.00	3	2	ABC Company
P035	Mini Projector	1,680.00	4	2	Digital World

Figure 1. The input data: the Inventory sheet in Excel (35 products).

Column	Meaning	Used by the program
Product ID	Unique code (P001 to P035)	Not used
Product Name	Name of the product	Yes, used as the key
Price	Unit price	Yes, for sales
Remaining Quantity	Units still in stock	Yes, for low-stock check
Quantity Sold	Units already sold	Yes, for sales
Customer	Who bought the product	Not used

03 / CODE

The Python script

The whole program lives in a single file, `Automation.py`, in about 40 lines.

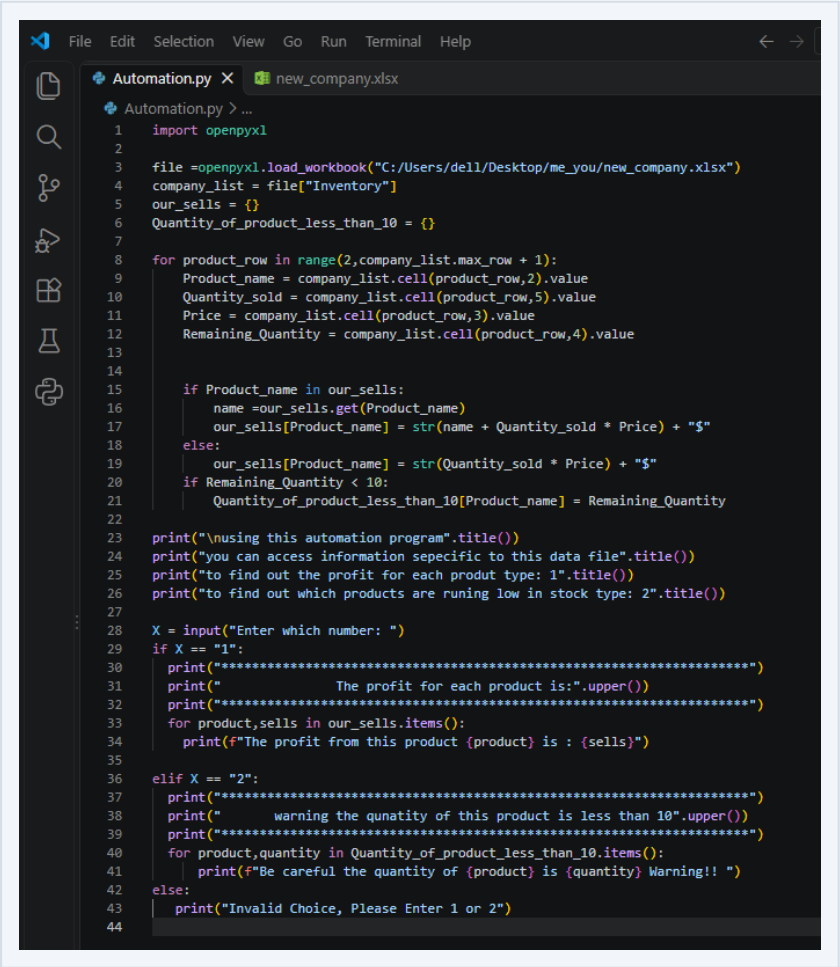


Figure 2. Automation.py in VS Code.

Reading the code

- **Lines 1 to 6:** import `openpyxl`, open the workbook and create two empty dictionaries to store results.
- **Lines 8 to 21:** loop over every row from row 2 to the last row, reading name, price, remaining and sold quantities.
- **Lines 15 to 19:** store `Price x Quantity Sold` for each product, followed by a `$` sign.
- **Lines 20 to 21:** if a product has fewer than 10 units left, save it in a second dictionary for the low-stock report.
- **Lines 23 to 26:** print the welcome message and the menu.
- **Lines 28 to 43:** read the user's choice with `input()` and print the right report with `if / elif / else`.

> **Good to know**
> Product names are used as dictionary keys, so each product gets its own result. Text formatting such as `.title()` and `.upper()` keeps the output neat.

04 / RUN

Starting the program

Running `Automation.py` shows a short menu. The user types **1** for sales or **2** for low stock.

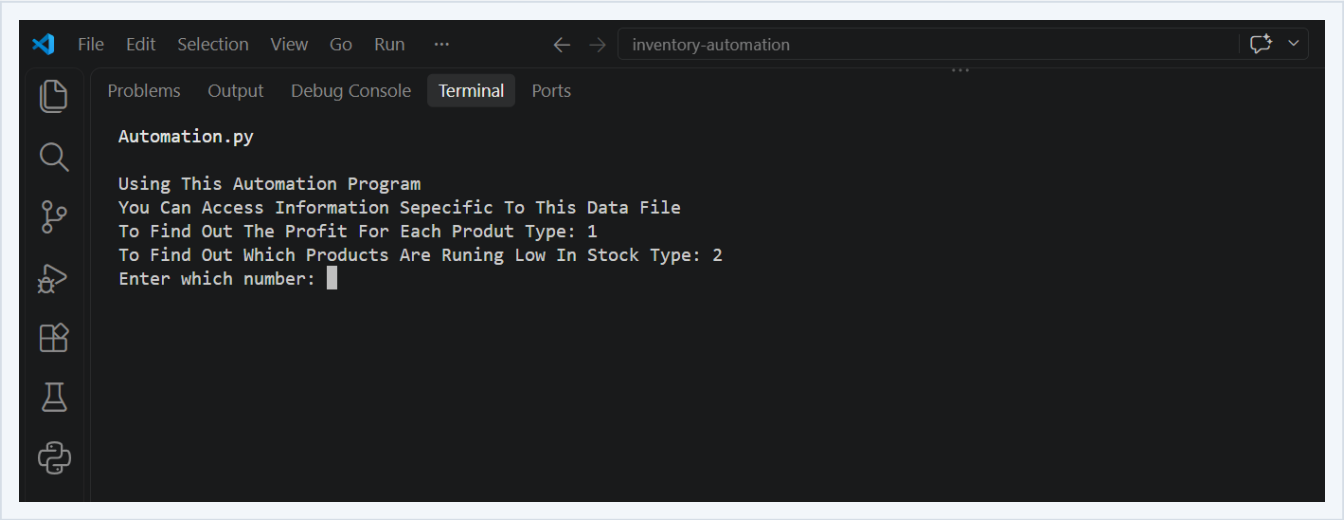


Figure 3. The main menu waiting for the user's choice.

Handling wrong input

If the user types anything other than 1 or 2, the program does not crash. It prints a clear message asking for a valid choice.

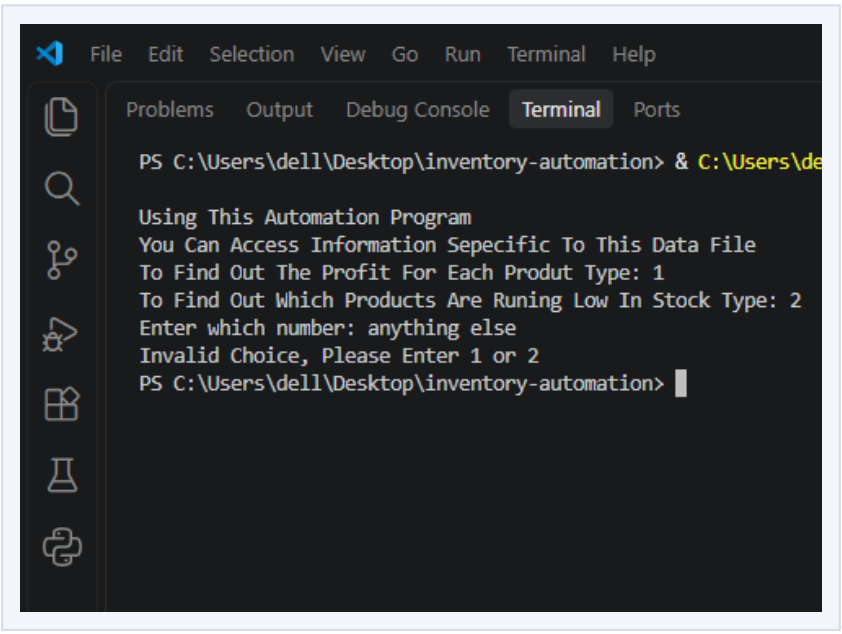


Figure 4. Entering an invalid value shows a friendly error message.

05 / OUTPUT 1

Sales per product

Choosing option 1 prints the total sales amount of every product in the file.

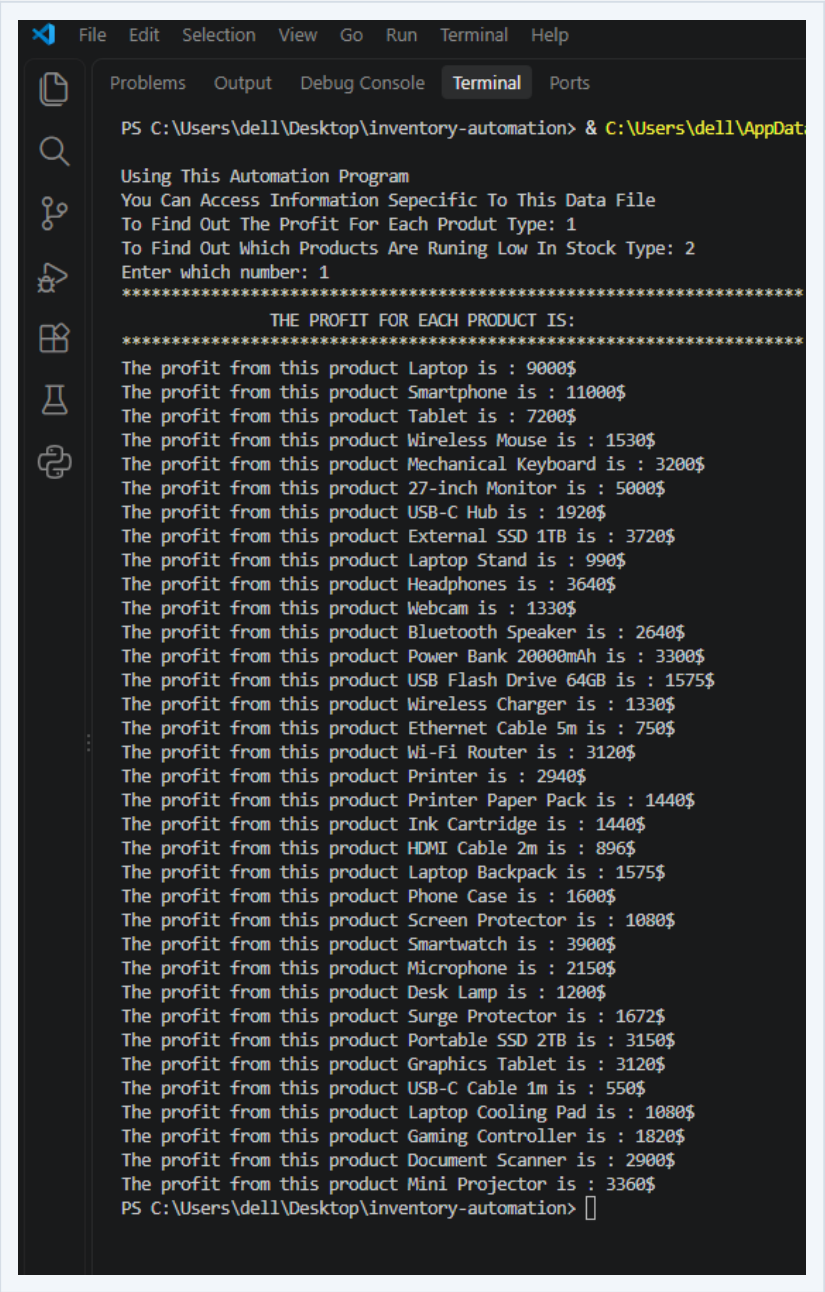


Figure 5. Option 1: result for all 35 products.

How to read it

Each line shows one product and its total, calculated as:

> **Formula**
> `Price x Quantity Sold`

Example from the data

- **Laptop:** 4,500 x 2 = **9,000$**
- **Smartphone:** 2,200 x 5 = **11,000$**
- **Tablet:** 1,800 x 4 = **7,200$**

The results match the Excel sheet exactly, and the Smartphone is the top earner in this dataset.

06 / OUTPUT 2

Low-stock warnings

Choosing option 2 lists every product with fewer than 10 units remaining.

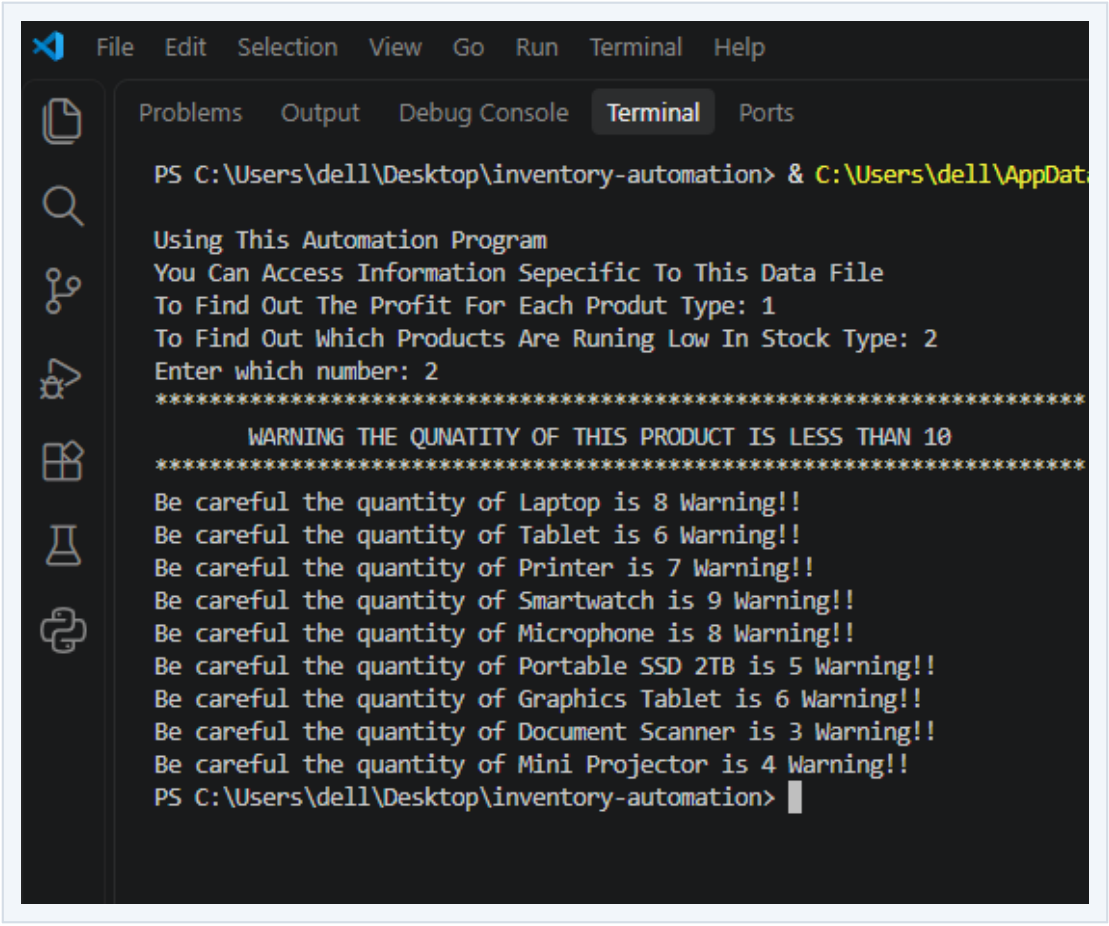


Figure 6. Option 2: 9 products need restocking.

Products to restock

Product	Left	Product	Left
Document Scanner	3	Smartwatch	9
Mini Projector	4	Laptop	8
Portable SSD 2TB	5	Microphone	8
Tablet	6	Printer	7
Graphics Tablet	6		

Why it matters
Instead of scanning 35 rows by eye, the owner sees at once which products are about to run out, and can reorder the most urgent ones first (Document Scanner, Mini Projector, Portable SSD 2TB).

07 / SUMMARY

Results and next steps

What this project demonstrates

- Reading and processing real spreadsheet data with Python and **openpyxl**.
- Using loops, dictionaries, conditions and f-strings to build a useful tool.
- Designing a simple, user-friendly terminal menu with input validation.
- Turning a repetitive manual task into an instant automated report.

Possible improvements

- Ask for the Excel file path instead of writing it inside the code.
- Make the low-stock threshold (currently 10) configurable by the user.
- Export the reports to a new Excel or CSV file.
- Add more reports, such as best-selling product or sales per customer.
- Let the menu repeat so several reports can be run in one session.

> **Project files**
> `Automation.py` | `new_company.xlsx` | README on GitHub